Operating Systems

Interview Preparation Guide

Top 20 Most Asked Questions & Answers
Intermediate to Advanced Level | With Easy Explanations

Level	Count	Topics Covered
Intermediate	12	Process, Threads, Scheduling, Deadlock Basics, Synchronization, Paging, Virtual Memory, File Systems
Advanced	8	Belady's Anomaly, Banker's Algorithm, Thrashing, Race Conditions, Kernel Design, Spooling, Fragmentation, Boot Process

Structured for quick revision before technical interviews

Table of Contents

Q1.	What is the difference between a Process and a Program?	Intermediate
Q2.	Explain the different states of a process (Process Life Cycle).	Intermediate
Q3.	What is a Thread, and how is it different from a Process?	Intermediate
Q4.	What is Context Switching and why does it happen?	Intermediate
Q5.	What are the main CPU Scheduling Algorithms?	Intermediate
Q6.	What is a Deadlock, and what are its 4 necessary conditions?	Intermediate
Q7.	How can Deadlocks be prevented, avoided, or detected?	Intermediate
Q8.	What is the difference between a Semaphore and a Mutex?	Intermediate
Q9.	What is Paging in Memory Management?	Intermediate
Q10.	What is Virtual Memory and why is it used?	Intermediate
Q11.	What is a Page Fault?	Intermediate
Q12.	What is a File System, and what are common File Allocation Methods?	Intermediate
Q13.	What is Belady's Anomaly?	Advanced
Q14.	Explain the Banker's Algorithm with its core idea.	Advanced
Q15.	What is Thrashing in Operating Systems?	Advanced
Q16.	What is a Race Condition, and what conditions must a Critical Section solution satisfy?	Advanced
Q17.	What is the difference between Monolithic Kernel and Microkernel?	Advanced
Q18.	What is Spooling, and how is it different from Buffering?	Advanced
Q19.	Explain Internal vs External Fragmentation.	Advanced
Q20.	What happens during the Operating System Boot Process?	Advanced

Q1 **Process vs Program** Intermediate

What is the difference between a Process and a Program?

ANSWER

A program is a passive entity — it's just a set of instructions stored on disk (an .exe or binary file). A process is an active entity — it's a program in execution, loaded into memory, with its own PCB (Process Control Block), CPU registers, program counter, and memory allocation (stack, heap, code, data segments).

EASY EXPLANATION: Easy way to remember: Program = recipe written on paper. Process = the recipe actually being cooked right now in the kitchen, with pots, ingredients and a chef (CPU) actively working on it.

Q2 **Process States** Intermediate

Explain the different states of a process (Process Life Cycle).

ANSWER

A process moves through these states during its life:

1. New - process is being created.
2. Ready - process is in memory, waiting for CPU time.
3. Running - instructions are being executed by the CPU.
4. Waiting/Blocked - process is waiting for an I/O operation or event to complete.
5. Terminated - process has finished execution and is removed from memory.

EASY EXPLANATION: Think of it like a student in an exam hall: New = entering the room, Ready = seated waiting for the exam to start, Running = actually writing the exam, Waiting = paused because they need more paper, Terminated = submitted the paper and left.

Q3 **Threads** Intermediate

What is a Thread, and how is it different from a Process?

ANSWER

A thread is the smallest unit of CPU execution within a process. A single process can have multiple threads that share the same memory space (code, data, files) but have their own stack, registers, and program counter.

Key difference: Processes are independent and have separate memory; threads within the same process share memory, which makes communication between threads faster but also riskier (data corruption if not synchronized).

EASY EXPLANATION: A process is like an office building. Threads are like employees working inside that same building — they share the same resources (printer, meeting rooms) but each does their own task independently.

Q4 **Context Switching** Intermediate

What is Context Switching and why does it happen?

ANSWER

Context switching is the process of saving the state (registers, program counter, memory info) of a currently running process/thread and loading the saved state of another one, so the CPU can switch between them.

It happens due to: interrupts, I/O requests, time-slice expiry (in time-sharing systems), or a higher priority process needing the CPU. It has an overhead cost since no useful work is done during the switch itself.

EASY EXPLANATION: Imagine you're reading a book (Process A) and someone calls you (interrupt). You bookmark your page (save state), attend the call (Process B runs), then come back and open the book to your bookmark (restore state).

Q5 **CPU Scheduling Algorithms** Intermediate

What are the main CPU Scheduling Algorithms?

ANSWER

1. FCFS (First Come First Serve) - processes executed in arrival order; simple but can cause the convoy effect.

2. SJF (Shortest Job First) - shortest burst-time process runs first; minimizes average waiting time but needs burst time prediction.

3. Priority Scheduling - process with highest priority runs first; can cause starvation for low priority processes.

4. Round Robin - each process gets a fixed time quantum in a cyclic order; fair and good for time-sharing systems.

5. Multilevel Queue Scheduling - processes are divided into different queues (e.g. foreground/background) each with its own scheduling algorithm.

EASY EXPLANATION: Think of a queue at a coffee shop: FCFS = first person in line served first. SJF = person with the smallest order served first. Round Robin = everyone gets 2 minutes of the barista's time, then the next person, cycling around.

Q6 **Deadlock** Intermediate

What is a Deadlock, and what are its 4 necessary conditions?

ANSWER

A deadlock is a situation where two or more processes are stuck forever, each waiting for a resource held by another process in the group, so none of them can proceed.

Four necessary conditions (Coffman conditions) for deadlock to occur:

1. Mutual Exclusion - only one process can use a resource at a time.

2. Hold and Wait - a process holds one resource while waiting for another.

3. No Preemption - resources cannot be forcibly taken away from a process.

4. Circular Wait - a closed chain of processes exists, each waiting for a resource held by the next.

EASY EXPLANATION: Imagine two cars meeting on a single-lane bridge from opposite ends. Each car has moved forward and neither can reverse or move forward without the other backing off. Both are stuck forever — that's a deadlock.

Q7 **Deadlock Handling** Intermediate

How can Deadlocks be prevented, avoided, or detected?

ANSWER

Prevention: break one of the 4 Coffman conditions (e.g., request all resources at once to avoid hold-and-wait).

Avoidance: use algorithms like the Banker's Algorithm, which checks if granting a resource keeps the system in a 'safe state' before allocating it.

Detection and Recovery: allow deadlocks to happen, detect them using a resource allocation graph / wait-for graph, then recover by killing a process or preempting resources.

EASY EXPLANATION: Prevention = never let the situation arise (strict traffic rules on the bridge). Avoidance = a traffic controller who checks before letting a car onto the bridge. Detection = let cars go, but have a system that spots a jam and calls a tow truck to fix it.

Q8 **Semaphore vs Mutex** Intermediate

What is the difference between a Semaphore and a Mutex?

ANSWER

A Mutex (Mutual Exclusion) is a locking mechanism that allows only ONE thread to access a critical section at a time — it's binary (locked/unlocked) and owned by the thread that locks it.

A Semaphore is a signaling mechanism that uses a counter to allow a fixed number of threads to access a resource. It can be binary (0/1, similar to mutex) or counting (allows N threads). Unlike a mutex, a semaphore can be signaled ('released') by a thread other than the one that acquired it.

EASY EXPLANATION: Mutex is like a single bathroom key — only one person can use the bathroom, and only that person can return the key. Semaphore is like a parking lot with 5 spots and a counter — up to 5 cars can park, and any car (not necessarily the same one) can leave, freeing a spot for someone else.

Q9 **Paging** Intermediate

What is Paging in Memory Management?

ANSWER

Paging is a memory management technique that divides a process's logical memory into fixed-size blocks called 'pages', and physical memory into blocks of the same size called 'frames'. Pages are mapped to frames using a page table, so a process doesn't need to be stored contiguously in physical memory.

This eliminates external fragmentation (no need for contiguous free memory) but can cause internal fragmentation (last page may not be fully used).

EASY EXPLANATION: Imagine a book split into equal-sized pages (chapters can be stored anywhere on a shelf, not necessarily in order). A table of contents (page table) tells you exactly where each page/chapter is physically located, so you can still read the book in order even though its pages are scattered.

Q10 **Virtual Memory** Intermediate

What is Virtual Memory and why is it used?

ANSWER

Virtual memory is a technique that gives each process the illusion of having a large, continuous block of memory, even if the physical RAM is smaller. It's implemented by keeping only actively used parts of a process in RAM, and the rest on disk (in a swap space/page file), moving pages in and out as needed.

Benefits: allows running programs larger than physical RAM, enables multiprogramming (more processes run simultaneously), and provides memory protection/isolation between processes.

EASY EXPLANATION: Think of virtual memory like a small desk (RAM) and a big storage cabinet (disk). You only keep the files you're actively working on, on the desk. Anything you're not using right now goes back into the cabinet, but it feels like your desk is huge because you can always pull out what you need.

Q11 **Page Fault** Intermediate

What is a Page Fault?

ANSWER

A page fault occurs when a process tries to access a page that is currently NOT loaded in physical memory (RAM). The OS then pauses the process, loads the required page from disk into a free frame in RAM (this is called 'page swapping' or 'demand paging'), updates the page table, and resumes the process.

EASY EXPLANATION: It's like reaching for a book on your desk, realizing it's actually in the storage cabinet downstairs, walking down to fetch it, and then continuing your work. That trip downstairs is the 'page fault' — it costs time, so we try to minimize how often it happens.

Q12 **File Systems** Intermediate

What is a File System, and what are common File Allocation Methods?

ANSWER

A file system is how the OS organizes, stores, retrieves, and manages data on a storage device (like a hard disk or SSD). It manages file names, directories, permissions, and physical storage location.

Common allocation methods: Contiguous Allocation (file stored in consecutive blocks - fast but causes fragmentation), Linked Allocation (each block points to the next - no fragmentation but slow random access), and Indexed Allocation (a separate index block stores pointers to all data blocks - good random access, used in modern systems like inodes in Linux).

EASY EXPLANATION: Think of a file system like a library's cataloging system — it doesn't just store books (data), it also keeps track of exactly where each book is, so you can find and retrieve it quickly instead of searching shelf by shelf.

Advanced Level Questions

Q13 **Belady's Anomaly** Advanced

What is Belady's Anomaly?

ANSWER

Belady's Anomaly is a counter-intuitive phenomenon where increasing the number of page frames (more memory) actually INCREASES the number of page faults, instead of decreasing them, for certain page replacement algorithms — most notably FIFO (First In First Out).

It does not occur in 'stack-based' algorithms like LRU (Least Recently Used) or Optimal Page Replacement, which is one reason LRU is generally preferred over FIFO.

EASY EXPLANATION: Normally, more shelf space (frames) should mean fewer trips to the cabinet (page faults). But with FIFO, sometimes adding more shelf space can actually cause MORE trips due to how old items get replaced — a surprising exception that interviewers love to test because it goes against intuition.

Q14 **Banker's Algorithm** Advanced

Explain the Banker's Algorithm with its core idea.

ANSWER

The Banker's Algorithm is a deadlock AVOIDANCE algorithm. Before granting a resource request, the OS simulates the allocation and checks if the system remains in a 'safe state' (i.e., there's still some order in which all processes can finish without deadlock).

It uses data structures: Available (free resources), Max (maximum demand of each process), Allocation (currently allocated), and Need (Max - Allocation). If granting a request keeps the system safe, it's approved; otherwise the process must wait.

EASY EXPLANATION: It works like a bank manager approving loans: before approving, they check if the bank still has enough money left to satisfy everyone's maximum possible future loan requests, in some order. If yes, approve the loan. If it would risk the bank running dry, they say wait.

Q15 **Thrashing** Advanced

What is Thrashing in Operating Systems?

ANSWER

Thrashing occurs when a system spends most of its time swapping pages in and out of memory (handling page faults) instead of executing actual instructions. It happens when too many processes are running with too little physical memory, causing each process to not have enough frames, leading to constant page faults.

Solutions: reduce the degree of multiprogramming (fewer processes running at once), use the Working Set Model to allocate frames based on a process's actual working set, or add more physical RAM.

EASY EXPLANATION: Imagine a worker with only 1 sq ft of desk space trying to work with 10 files. They constantly put one file away to get another, spending all their time filing instead of actually working. That's thrashing — the system is 'busy' but getting almost nothing productive done.

Q16 **Race Condition & Critical Section** Advanced

What is a Race Condition, and what conditions must a Critical Section solution satisfy?

ANSWER

A race condition occurs when multiple processes/threads access and modify shared data concurrently, and the final result depends on the unpredictable timing/order of execution — leading to inconsistent or incorrect results.

A valid solution to the Critical Section Problem must satisfy 3 conditions: Mutual Exclusion (only one process in the critical section at a time), Progress (if no process is in the critical section, one of the processes waiting must be allowed in without indefinite delay), and Bounded Waiting (there's a limit on how many times other processes can enter before a waiting process gets its turn, avoiding starvation).

EASY EXPLANATION: Imagine two people editing the same shared Google Doc line at the exact same time without any lock — whoever's change saves last overwrites the other's, and the final content depends on split-second timing. That's a race condition. A 'critical section' rule (like taking turns) prevents this mess.

Q17 **Monolithic vs Microkernel** Advanced

What is the difference between Monolithic Kernel and Microkernel?

ANSWER

A Monolithic Kernel runs the entire OS (process management, memory management, file systems, device drivers) as a single large program in kernel space. It's fast (no communication overhead between components) but less stable — a bug in one driver can crash the whole system. Example: Linux.

A Microkernel keeps only the bare minimum in kernel space (IPC, basic scheduling, basic memory management) and runs other services (file systems, drivers) as separate user-space processes. It's more stable and secure (a crashing driver doesn't crash the OS) but slower due to inter-process communication overhead. Example: QNX, and (partially) Windows NT-based kernels.

EASY EXPLANATION: Monolithic kernel = one huge Swiss Army knife where everything is welded into a single tool — fast to use but if one blade breaks, you might need to replace the whole tool. Microkernel = a toolbox with separate small tools — slightly slower to grab each one, but if one tool breaks, the rest still work fine.

Q18 **Spooling** Advanced

What is Spooling, and how is it different from Buffering?

ANSWER

Spooling (Simultaneous Peripheral Operations On-Line) is a technique where data for slow I/O devices (like a printer) is temporarily stored in a queue/disk buffer, allowing a process to continue running instead of waiting for the slow device. Multiple jobs can queue up and be processed in order, independent of CPU speed.

Buffering, on the other hand, is used to handle mismatches in data transfer speed between two devices (e.g., CPU and disk) by temporarily holding data in a memory area — but it typically handles ONE job/stream at a time, while spooling manages a QUEUE of jobs and uses disk (not just memory) as storage.

EASY EXPLANATION: Spooling is like a printer queue at an office — everyone sends their print jobs, and they wait in line on the server even though the printer is slow, so no one has to physically stand around waiting for their turn.

Q19 **Fragmentation Types** Advanced

Explain Internal vs External Fragmentation.

ANSWER

Internal Fragmentation occurs when memory is allocated in fixed-size blocks (like pages or fixed partitions), but the process doesn't use the entire block — the leftover, unused space WITHIN the allocated block is wasted and can't be given to another process.

External Fragmentation occurs when free memory is scattered in small, non-contiguous chunks throughout memory (common in variable-sized partitioning / contiguous allocation). Even if the total free memory is enough, it can't be used because no single free chunk is large enough for a new process.

EASY EXPLANATION: Internal fragmentation is like buying a 12-egg carton but only needing 10 eggs — the 2 empty slots in YOUR carton are wasted and can't help anyone else. External fragmentation is like having enough empty seats scattered across a movie theater for a group of 5 friends, but no 5 SEATS TOGETHER — so the group still can't sit together even though enough total seats exist.

Q20 **Bootstrapping / Boot Process** Advanced

What happens during the Operating System Boot Process?

ANSWER

1. Power On -> the CPU starts executing firmware code, either the BIOS (Basic Input/Output System) or the more modern UEFI, stored in ROM/flash memory.

2. POST (Power-On Self Test) -> firmware checks hardware (RAM, keyboard, disk) for basic faults.

3. Bootloader -> firmware locates and loads a small program called the bootloader (e.g., GRUB) from the boot device (disk/SSD).

4. Kernel Load -> the bootloader loads the OS kernel into memory and transfers control to it.

5. Kernel Initialization -> the kernel initializes device drivers, mounts the root file system, and starts the first user-space process (like 'init' or 'systemd' on Linux), which then starts all other system services.

EASY EXPLANATION: It's like starting a car: turn the key (power on), the car does a quick dashboard self-check (POST), the starter motor gets the engine going (bootloader), and finally the engine runs on

its own and takes over everything (kernel takes control and starts all normal operations).